

Topic 6.1

6.1: Comments on Reading Assignments for Module 5

The primary focus of this module is the topic of inheritance, wherein we derive new classes from already existing classes, preserving some or all of the old functionality, and adding new functionality as needed.

With inheritance we see the *protected* scope declaration. *Thinking in C++* argues against use of protected data members, preferring protected or public member functions instead. From a pure software engineering perspective (ease of debugging, using alternate physical implementations, etc.) this is appropriate. However, it is often the case that an entire hierarchy is designed and written as one module. In such cases it is reasonably common to see protected data members.

I have included Chapter 17, on Debugging and Optimization, as an optional reading exercise. The debugging portion of this chapter is quite useful, although the author gets somewhat hung up on optimization. It's important to realize that in today's world, it is *people* who are expensive. Computers are cheap. Programs should be written for *maximum maintainability*. Worrying about optimization too early or too often makes it more difficult to maintain code.

It is true that programs need to be reasonably efficient. However, we apply the 80/20 rule, which says that 20% of the lines of code account for 80% of the execution time, that is, the mainline code. The other 80% of the code is for user-friendly interfaces, error recovery, etc. If you worry about optimizing the wrong parts of the code, you end up with an unmaintainable mess that is only marginally more efficient than the unoptimized yet maintainable version. Tools known as *profilers* (no, not the FBI agent on Saturday nights) enable you to see where your program is spending all its time. Those are the parts to optimize. There typically aren't very many of them.

Topic 6.2 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-6-dot-2>)